

## 一、这是一套“全新”的算法吗？

坦诚地说，从纯粹的计算机科学理论来看，这不是一个“从零发明的全新数学算法”。它属于一种高度创新的复合系统架构(**Systems Engineering**)。

它巧妙地将经典的计算机科学算法(如 Dijkstra、Levenshtein)、地理信息科学(GIS)公式和现代大语言模型(LLM)结合在了一起。在学术界和工业界，这种将基础算法进行“场景化改造”以解决特定长尾问题(比如针对视障人群规避陡坡，或者处理极具地方口音的错别字)的做法，本身就是极具价值的创新。招生官和评委看重的正是你这种发现痛点、重组技术栈并最终落地的工程能力。

---

## 二、算法拆解与系统性学习路径

既然你希望系统性地学习这块内容，我们可以按照你文档中的四个核心步骤进行拆解，并为你提供学习方向：

### 1. NLP与意图识别 (字符串匹配与哈希)

- 算法本质:处理非结构化和带有噪音的文本输入。系统使用了别名映射和模糊字符串匹配。
- 学习重点:
  - 哈希表 (**Hash Tables/Dictionaries**):理解  $O(1)$  时间复杂度的键值对查找，这是别名映射表的基础。
  - 动态规划 (**Dynamic Programming**):重点学习计算编辑距离(Levenshtein Distance)的矩阵推导过程。\* 正则表达式 (**Regex**) 与字符串清洗:掌握如何在Python中高效地剥离标点和提取特征。

### 2. 加权空间图谱与路径规划 (图论算法)

- 算法本质:在离散的节点网络中寻找最优解。结合你的建筑与空间规划背景，这部分理解起来会非常直观。
- 学习重点:
  - 图的表示 (**Graph Representation**):学习邻接矩阵和邻接表。熟悉 Python 中的 NetworkX 库。
  - **Dijkstra** 算法:深入理解其贪心策略，以及如何使用优先队列(Min-Heap)将其时间复杂度优化至  $O(E + V \log V)$ 。\* A (A-Star) 启发式搜索\*:这是 Dijkstra 的进阶版。建议你拓展学习 A\* 算法，它在寻路时引入了方向引导，计算速度更快，在游戏开发(如智能体寻路)中极其常用。

### 3. 矢量几何方向计算 (计算几何)

- 算法本质: 将地球表面的经纬度转化为人类可感知的局部坐标系和偏航角。
- 学习重点:
  - **Haversine** 公式: 了解如何利用球面三角学计算地球上两点间的真实距离。
  - 向量点乘与叉乘 (**Dot & Cross Product**): 用于判断一个节点在另一个节点的“左侧”还是“右侧”。
  - 方位角与偏航角 (**Bearing & Yaw**): 掌握如何将绝对坐标系转化为相对坐标系。

### 4. LLM 语义重构 (提示词工程)

- 算法本质: 将结构化数据 (JSON/图节点) 转化为自然语言, 并解决模型幻觉。
- 学习重点: 深入研究 Prompt Engineering, 特别是 Few-Shot Prompting (少样本提示) 和 Chain of Thought (思维链), 确保 AI 能稳定输出“左转、右转”的逐个路口指令, 而不遗漏细节。

---

## 三、下一步: 如何证明算法的有效性?

你提到的两种验证方法 (数据分析图 vs. Unity 智能体) 都非常棒, 但它们的目的是适用场景不同。作为一个完整的大创或作品集项目, 我强烈建议你两者结合, 分步实施:

#### 第一阶段: Python 脚本模拟 1000 次请求 (验证鲁棒性)

- 做法: 写一个 Python 脚本, 自动生成 1000 个包含拼音错误、别名或错别字的测试用例 (比如模拟不同程度的“烫嘴普通话”) 输入到你的系统中。
- 目的: 生成混淆矩阵和准确率图表。这能用硬核的数据证明你的“多级容错意图识别”引擎比直接调用百度/高德 API 的容错率高多少。
- 优势: 实现成本极低, 只需要跑几分钟的代码, 生成的数据图表放在 PPT 或 CV 里极具说服力。

#### 第二阶段: Unity 生成智能体 A/B 测试 (验证空间逻辑与体验)

- 做法: 在 Unity 中搭建一个小型场景 (例如鼓浪屿某个复杂街区的简化白模)。设置 A 组 Agent 使用传统最短路径算法; B 组 Agent 接入你的算法, 根据地形坡度 (针对特殊群体) 和生成的指令来行走。\* 目的: 直观展示算法的“人文关怀”和“动态导引”。你可以录制一段视频: A 组智能体卡在陡坡或者频繁转向, 而 B 组智能体通过 `get_safest_route` 平稳且安全地到达终点。

- 优势:视觉冲击力极强。如果你未来申请的方向偏向游戏开发或交互设计,这个 Unity Demo 将会是你作品集中最吸睛的亮点。

总结来说,先用 1000 次 Python 模拟数据来证明“算法听得懂”(NLP 层面),再用 Unity 仿真视频来证明“算法带得好”(物理与空间层面)。